

Flutter, React Admin, FastAPI, MongoDB Atlas, AWS S3, Firebase Auth/FCM, Docker, GitHub Actions, AWS, Gemini/OpenAI, RAG and OCR, with a modular monolith direction, repository pattern, service layer and SOLID principles. The product scope includes AI health chat, health report memory/analyzer, medicine reminders, water reminders, goals, daily summaries, personalized workout/nutrition support and RAG-based answers from user health history.

First: Blumetara AI core requirements

Blumetara is not just a normal CRUD app. It has four types of workload:

1. **Normal app workload**
Login, profile, dashboard, goals, reminders, subscriptions, admin dashboard.
2. **Medical file workload**
Uploading PDFs, prescriptions, blood reports, health images, storing metadata, processing OCR, storing files securely.
3. **AI/RAG workload**
User asks health questions, app retrieves their profile + report chunks + goal history, sends controlled context to Gemini/OpenAI, and returns a response.
4. **Background workload**
Medicine reminders, water reminders, daily summaries, weekly insights, report processing, embeddings, push notifications.

Because this is health/medical data, every design must include:

- Firebase Auth + backend JWT verification.
- Role-based access control.
- Encrypted storage.
- Audit logs.
- No raw health reports in Redis.
- MongoDB/S3 as systems of record.
- S3 for files, MongoDB for metadata.
- RAG filtering by `userId`, `familyId`, and `reportId`.
- Backups and restore plan.
- Monitoring and failure handling.

Model 1: Simple Production Modular Monolith

Best for: MVP + Version 1

This is the simplest serious architecture. It is not a toy system. It is a **production-aware modular monolith**.

Core idea

You build **one FastAPI backend**, but inside the backend, the code is cleanly separated into modules:

- Auth Module
- User/Profile Module
- AI Chat Module
- Health Reports Module
- Goals Module
- Reminders Module
- Nutrition Module
- Workout Module
- Notification Module
- Subscription Module
- Admin Module
- Audit Log Module

So from deployment point of view, it is one backend.

From code structure point of view, it behaves like a clean professional system.

Main architecture

Client side:

- Flutter mobile app
- React admin dashboard

Entry layer:

- Cloudflare
- AWS WAF
- Application Load Balancer

Backend:

- FastAPI app deployed on AWS App Runner or ECS Fargate
- Internal modular monolith

- Background workers for OCR, reminders, embeddings, summaries

Data:

- MongoDB Atlas for user data, profile, goals, chats, reminders, reports metadata
- AWS S3 for health report PDFs/images
- MongoDB Atlas Vector Search or vector collection for embeddings
- Redis/ElastiCache for cache, rate limits, locks

External services:

- Firebase Auth
- Firebase Cloud Messaging
- Gemini/OpenAI
- Payment gateway
- Email service

What it supports

This model can support:

- MVP
- Version 1
- Basic subscriptions
- AI chat
- RAG
- OCR
- Report upload
- Medicine reminders
- Water reminders
- Goal tracking
- Daily summaries
- Food/workout plans
- Admin dashboard

This is enough for your first real launch.

Redis usage in Model 1

Redis is used only for fast temporary data.

Good Redis usage:

```
rate:user:{userId}:ai_chat:{yyyyMMddHHmm}  
profile:{userId}:summary:v1  
lock:reminder:{reminderId}:{scheduledAt}
```

idempotency:upload:{userId}:{requestId}
admin:metrics:daily:{date}
provider:llm:{provider}:status

Do **not** store these in Redis:

full health reports
raw OCR text
full AI prompts with medical data
full AI responses with medical history
access tokens
payment card data
cross-user RAG results

Redis is not the database. Redis is just a performance and safety helper.

High availability

Minimum serious setup:

- FastAPI running in at least **2 Availability Zones**
- ALB health checks
- MongoDB Atlas replica set
- S3 versioning enabled
- Redis ElastiCache with replica in production
- At least 2 background workers
- CloudWatch monitoring
- Backups enabled

Strengths

- Fastest to build.
- Cheapest to operate.
- Easy to debug.
- Best for small team.
- Clean enough for future growth.
- Best architecture for MVP + V1.

Weaknesses

- All backend modules deploy together.
- AI traffic, admin traffic and normal app traffic share the same backend.
- Scaling is less fine-grained.
- If the backend becomes huge, it can become harder to maintain later.

My view

This is the best **launch model**.

For your meeting, this sounds mature:

“We are starting with a high-availability modular monolith. This gives us production reliability without overengineering the MVP.”

Model 2: Hybrid Async Modular Platform

Best for: MVP + Version 1 + easier expansion for Version 2

This is the middle architecture. It is more advanced than Model 1, but not as heavy as full microservices.

Core idea

You still keep FastAPI mostly as a modular backend, but you separate **heavy background work** using queues and workers.

In Model 1, some async jobs may use Celery/Redis.

In Model 2, async jobs become more reliable using:

- AWS SQS
- EventBridge
- Dead-letter queues
- Worker fleet
- Outbox pattern

This makes the system much easier to expand later.

Main architecture

Client side:

- Flutter mobile app
- React admin dashboard

Entry layer:

- Cloudflare

- AWS WAF
- ALB

Backend:

- FastAPI modular backend
- Clear service layer
- AI Orchestrator module
- Report Processing module
- Reminder module
- Notification module

Async layer:

- SQS queues
- EventBridge events
- Worker services
- Dead-letter queues
- Retry logic

Data:

- MongoDB Atlas
- AWS S3
- Redis/ElastiCache
- Vector search
- Optional analytics read model later

External:

- Firebase Auth/FCM
- Gemini/OpenAI
- OCR provider
- Payment gateway
- Email service

What changes compared to Model 1

In Model 1, the backend may directly trigger background tasks.

In Model 2, the backend publishes events.

Example:

User uploads report

→ FastAPI saves metadata

- FastAPI publishes ReportUploaded event
- OCR worker consumes event
- OCR worker extracts text
- Embedding worker chunks + embeds
- Summary worker creates report summary
- Notification worker informs user

This is much more reliable.

Event examples

ReportUploaded
OCRCompleted
EmbeddingCompleted
ReminderDue
NotificationRequested
PaymentWebhookReceived
DailySummaryRequested
GoalProgressUpdated

Why this is better for expansion

Because future features can plug into events.

Example:

Later, when you add weekly insights:

GoalProgressUpdated
→ Insights worker listens
→ Generates weekly gap analysis

Later, when you add family accounts:

CriticalReminderMissed
→ FamilyAlert worker listens
→ Sends alert to family member

You do not need to rewrite the whole system.

Redis usage in Model 2

Redis becomes more focused:

Use Redis for:

rate limits
short-lived profile cache
distributed locks
idempotency keys
provider health
admin dashboard cache

Use SQS/EventBridge for:

OCR jobs
embedding jobs
summary jobs
notification jobs
payment webhook processing
daily summary generation

This is better because Redis is not a durable queue. SQS/EventBridge is safer for important background workflows.

High availability

Model 2 has stronger HA than Model 1:

- Backend across 2+ AZs
- MongoDB Atlas replica set
- S3 versioning
- ElastiCache Redis replication
- SQS durable queues
- Dead-letter queues for failed jobs
- Workers can scale independently
- Report processing does not block app usage
- Notification failure does not crash backend
- LLM outage does not break non-AI features

Strengths

- Still realistic to build.
- Much better async reliability.
- Easier to expand into future versions.
- Good for medical report processing.
- Better failure isolation than Model 1.
- Stronger story for investors.

Weaknesses

- More moving parts.
- More AWS configuration.
- More monitoring needed.
- Slightly more expensive.
- Debugging async flows is harder.

My view

This is the best **serious technical architecture** if you want to impress stakeholders without jumping too early into full microservices.

For your meeting, this sounds strong:

“Our recommended growth architecture is a hybrid async modular platform. The user-facing app remains simple and fast, while heavy healthcare workloads like OCR, embeddings, reminders and summaries run through durable background pipelines.”

Model 3: Managed AI Platform / Vertex AI-Style Architecture

Best for: premium future platform, AI-heavy scaling, investor roadmap

This is the most advanced and expensive option.

You mentioned **Vertex AI**, and yes, that can be a valid future design. But we need to be careful: your current confirmed stack is mainly AWS + MongoDB + Firebase + Gemini/OpenAI. Moving to Vertex AI means you are either doing **multi-cloud** or shifting the AI/data layer to Google Cloud.

There are two ways to do this.

Option 3A: AWS app layer + Google Vertex AI layer

This means:

- App backend remains on AWS.
- Files may stay in S3.
- MongoDB Atlas remains database.
- AI/RAG/OCR uses Google Cloud services.

AI layer can use:

- Vertex AI Gemini models
- Vertex AI embeddings
- Vertex AI Vector Search
- Document AI for OCR
- Cloud Pub/Sub for AI processing events
- Cloud Run for AI workers

This gives strong AI capabilities but creates cross-cloud complexity.

Option 3B: Full Google Cloud AI-native architecture

This means:

- FastAPI on Cloud Run or GKE
- Cloud Load Balancing
- Cloud Armor instead of AWS WAF
- Cloud Storage instead of S3
- Vertex AI for Gemini/embeddings
- Vertex AI Vector Search
- Document AI for OCR
- Pub/Sub for events
- Cloud Tasks / Cloud Scheduler
- Firebase Auth/FCM stays natural because Firebase is Google ecosystem

This is cleaner if you want to go all-in on Google Cloud.

Main architecture

Client side:

- Flutter app
- React admin dashboard

Entry layer:

- Cloudflare or Cloud Load Balancer
- Cloud Armor / WAF equivalent

Backend:

- FastAPI on Cloud Run or GKE
- API layer / BFF
- AI Orchestrator service

- Report service
- Reminder service
- Admin service

AI platform:

- Vertex AI Gemini
- Vertex AI Embeddings
- Vertex AI Vector Search
- Document AI OCR
- Vertex AI evaluation/monitoring if needed

Data:

- MongoDB Atlas can still be used
- Or Google Firestore/Cloud SQL, but that would be a bigger stack change
- Cloud Storage or S3 depending on cloud decision

Async:

- Pub/Sub
- Cloud Tasks
- Cloud Scheduler

Observability:

- Google Cloud Logging
- Google Cloud Monitoring
- Error Reporting
- Trace

What this supports

This model is best for:

- AI-heavy future versions
- Better managed OCR
- Better managed embeddings
- Better AI experiment tracking
- Easier Gemini-native integration
- Future AI agents
- More scalable RAG pipeline
- Better model monitoring and evaluation

Why it may be attractive

Vertex AI can make the AI side look more enterprise-grade.

For investors, this sounds strong:

“The long-term AI roadmap can move toward a managed AI platform where OCR, embeddings, retrieval, model routing and evaluation are handled through a dedicated AI infrastructure layer.”

But the tradeoff is important

This model is expensive and complex.

Problems:

- Multi-cloud networking if backend stays on AWS.
- More vendor complexity.
- Higher AI platform cost.
- More DevOps learning.
- More IAM/security setup.
- More difficult debugging across AWS + GCP.
- Bigger migration effort.

My view

This should **not** be your MVP architecture.

This should be presented as:

“Premium AI scaling option after product validation.”

Not as:

“We will build this first.”

For your actual launch, this is too much unless the client has strong funding and wants AI infrastructure as a major differentiator from day one.

Side-by-side comparison

Criteria	Model 1: Simple Modular Monolith	Model 2: Hybrid Async Modular Platform	Model 3: Vertex AI / Managed AI Platform
----------	-------------------------------------	---	---

Best for	MVP + V1	MVP + V1 + V2 expansion	Premium AI-heavy future
Build speed	Fastest	Medium	Slowest
Cost	Lowest	Medium	Highest
Complexity	Low-medium	Medium-high	High
Team size	1–3 devs	2–5 devs	4+ devs with cloud/AI experience
MVP suitability	Excellent	Good, but slightly heavier	Not ideal
Future scaling	Good	Very good	Excellent for AI workloads
HA story	Good	Strong	Strong, but more complex
Async processing	Basic workers	Durable queues/events	Managed cloud-native events
AI/RAG maturity	Good enough	Strong	Very strong
Debugging	Easiest	Moderate	Hardest
Investor appeal	Good	Very good	Very high, but must justify cost
Risk	Low	Medium	High if started too early

My clear recommendation

Use these 3 designs in this order:

Pitch Model 1 as the launch architecture

This is the practical one.

Say:

“For launch, we will use a high-availability modular monolith. It supports MVP and Version 1, keeps cost controlled, and gives us proper security, backups, Redis caching, RAG, OCR and background workers without unnecessary microservice complexity.”

Pitch Model 2 as the recommended growth architecture

This is the one I personally like the most for your stakeholder deck.

Say:

“As usage grows, we move to a hybrid async modular platform. Report processing, embeddings, reminders, daily summaries and notifications move to durable event-driven workflows. This gives us better reliability and expansion without rewriting the core product.”

Pitch Model 3 as the premium AI roadmap

This is for future/investor ambition.

Say:

“For a later AI-heavy version, we can introduce a managed AI platform such as Vertex AI for OCR, embeddings, vector search, model routing and AI evaluation. This is more expensive, so we should adopt it only after product validation or investor funding.”

Final architecture naming

Use these names in the presentation:

1. **Design 1 — Simple HA Modular Monolith**
 - MVP + Version 1
 - Cheapest and fastest
 - Best launch option
2. **Design 2 — Hybrid Async Modular Platform**
 - MVP + Version 1 + easier future expansion
 - Best overall technical recommendation
 - Stronger async and reliability
3. **Design 3 — Managed AI Platform with Vertex AI**

- Premium future roadmap
- Best AI scalability
- Expensive but investor-friendly

For the next step, we should create the **proper written architecture for Design 1 first**, then Design 2, then Design 3, and only after that generate diagrams.

can you please give me a google docs text format that i can copy paste it in google docs
Thought for 14s

Absolutely. Copy the text below directly into Google Docs. After pasting, apply **Heading 1 / Heading 2 / Heading 3** styles if needed.

BLUMETARA AI SYSTEM DESIGN OPTIONS

Architecture Concepts, Requirements, Tradeoffs and Recommendation

Project: Blumetara AI

Purpose: Stakeholder / Investor System Design Planning

Prepared For: Medical Tracking and AI Health Assistant Application

Current Stage: Architecture discussion before final diagram and PDF creation

1. Executive Summary

Blumetara AI is an AI-powered personal health tracking and medical assistant platform. The application helps users store health information, upload and understand medical reports, ask health-related questions, track goals, receive reminders, and get personalized health insights.

The confirmed product direction includes:

- Flutter mobile application
- React admin dashboard
- FastAPI backend
- MongoDB Atlas database

- AWS S3 for medical file storage
- Firebase Authentication
- Firebase Cloud Messaging for notifications
- Docker
- GitHub Actions
- AWS deployment
- Gemini/OpenAI for AI chat, OCR, RAG, and report understanding

Because this is a health and medical data application, the architecture must be designed with serious attention to:

- Data protection
- Encryption
- Audit logs
- Role-based access control
- Secure file handling
- High availability
- Backup and disaster recovery
- Safe AI usage
- Clear separation between cached data and sensitive medical records

This document compares three system design models for Blumetara AI:

1. **Design 1 — Simple High-Availability Modular Monolith**
Best for MVP + Version 1.
2. **Design 2 — Hybrid Async Modular Platform**
Best for MVP + Version 1, while making future expansion easier.
3. **Design 3 — Managed AI Platform with Vertex AI / Advanced AI Infrastructure**
Best for future AI-heavy scaling, investor roadmap, and premium expansion.

The recommendation is to launch with **Design 1**, prepare the backend structure so it can evolve into **Design 2**, and keep **Design 3** as the long-term AI scaling roadmap.

2. Core Product Requirements

Blumetara AI is not a simple CRUD application. It has multiple workload types.

2.1 User Application Workload

This includes normal user-facing product features:

- User registration and login
- User profile management
- Health profile management
- Dashboard
- Goal tracking
- Medicine reminders
- Water reminders
- Workout plans
- Nutrition tracking
- Subscription plans
- Admin dashboard

These features require fast APIs, reliable database access, authentication, and authorization.

2.2 Medical File Workload

Users may upload:

- Blood reports
- Medical reports
- Prescriptions
- Food images
- Profile images
- Health-related documents

These files should not be stored directly inside MongoDB. Large files should be stored in object storage such as AWS S3. MongoDB should store only metadata such as:

- File name
- File type
- S3 URL or object key
- Upload status
- Processing status
- User ID
- Report ID
- OCR status
- Summary status
- Created date
- Updated date

2.3 AI and RAG Workload

The AI system should answer user questions based on:

- User profile
- Health reports
- Past medical history
- Goals
- Reminders
- Food logs
- Workout history
- Report summaries
- Relevant document chunks

The AI flow should be:

User question

- Backend receives request
- User is authenticated
- Relevant profile and report context is retrieved
- Vector search finds relevant chunks
- Prompt builder creates controlled prompt
- Gemini/OpenAI generates response
- Response is returned to user
- Conversation is stored safely

2.4 Background Processing Workload

Some tasks should not block the user request. These should run in the background:

- OCR processing
- Report parsing
- Embedding generation
- AI report summaries
- Daily summaries
- Weekly insights
- Medicine reminder dispatch
- Water reminder dispatch
- Push notifications
- Payment webhook handling
- Email notifications
- Analytics aggregation

This is why the system needs workers, queues, or scheduled tasks.

3. Non-Functional Requirements

3.1 Availability

Since Blumetara AI handles health-related tracking, the system should avoid single points of failure wherever practical.

Important availability requirements:

- Backend should run across multiple Availability Zones.
- Load balancer should route traffic only to healthy backend instances.
- MongoDB Atlas should use replica sets.
- S3 should be used for durable file storage.
- Redis should not be the only place where important data exists.
- Workers should retry failed background jobs.
- Monitoring should detect failures early.

3.2 Security

The system must be secure by default.

Security requirements:

- Firebase Authentication for user login.
- Backend JWT verification for every protected API.
- Role-based access control.
- HTTPS everywhere.
- Encryption at rest.
- Encryption in transit.
- Secrets stored in AWS Secrets Manager or a secure equivalent.
- No API keys in frontend code.
- No secrets committed to GitHub.
- Audit logging for sensitive actions.
- Admin actions must be logged.
- Medical report access must be controlled.

3.3 Compliance-Aware Design

This document does not certify HIPAA, GDPR, or India DPDP compliance. However, the system should be designed in a compliance-aware way.

Compliance-aware controls:

- User consent for data storage and AI usage.
- Clear data deletion policy.
- Data retention policy.
- Audit logs.
- Encryption.
- Access control.
- Vendor review for Firebase, MongoDB Atlas, AWS, Gemini/OpenAI.
- Avoid logging sensitive health information.
- Avoid sending unnecessary medical data to LLM providers.

3.4 Scalability

The system should scale gradually.

The architecture should support:

- 100 users
- 1,000 users
- 10,000 users
- 100,000 users
- Future scale beyond that

But the MVP should not be overengineered from day one.

3.5 Maintainability

The backend should follow:

- Modular Monolith structure
- Clean Architecture
- Service Layer
- Repository Pattern
- SOLID principles
- Clear folder structure
- DTO/schema validation
- Dependency injection where useful
- Proper testing

4. Shared Technology Stack

The three designs share the same base product stack.

4.1 Frontend

Mobile app:

- Flutter
- Android
- iOS

Admin dashboard:

- React
- Next.js if needed
- Role-based admin access

4.2 Backend

Backend framework:

- FastAPI
- Python
- REST APIs
- Async workers where needed

4.3 Database

Primary database:

- MongoDB Atlas

Used for:

- Users
- Profiles
- Conversations
- Messages
- Goals
- Goal progress
- Health reports metadata
- Food logs
- Workout plans
- Medicines
- Reminders

- Notifications
- Subscriptions
- Audit logs
- Family members

4.4 File Storage

Object storage:

- AWS S3

Used for:

- Medical report PDFs
- Prescription images
- Food images
- Profile images
- Voice files
- Health documents

4.5 Authentication and Notifications

Authentication:

- Firebase Auth

Push notifications:

- Firebase Cloud Messaging

4.6 AI Layer

AI providers:

- Gemini
- OpenAI

AI capabilities:

- Chat completion
- Report understanding
- OCR assistance
- Embeddings
- RAG

- Summaries
- Personalized health responses

4.7 DevOps

DevOps tools:

- Docker
 - GitHub Actions
 - AWS
 - CloudWatch
 - Secrets Manager
 - ECR
 - App Runner or ECS Fargate
-

5. Design 1 — Simple High-Availability Modular Monolith

5.1 Purpose

Design 1 is the simplest serious production architecture for Blumetara AI.

It is designed to support:

- MVP
- Version 1
- Early users
- First paid customers
- Initial investor/stakeholder demo
- Controlled cost
- Faster development

This is the recommended launch architecture.

5.2 Core Idea

The system uses one FastAPI backend application, but the backend is internally divided into clean modules.

This means there is one deployable backend, but the code is not messy. It is structured like a professional system.

5.3 Backend Modules

The FastAPI backend should contain these modules:

- Authentication Module
- User Profile Module
- AI Chat Module
- Health Report Module
- Goal Tracking Module
- Nutrition Module
- Workout Module
- Reminder Module
- Notification Module
- Subscription Module
- Admin Module
- Analytics Module
- Audit Log Module

Each module should have:

- API routes
- Request/response schemas
- Service layer
- Repository layer
- Validation rules
- Error handling
- Unit tests

5.4 High-Level Architecture

The architecture flow is:

Flutter Mobile App / React Admin Dashboard

- Cloudflare
- AWS WAF
- Application Load Balancer
- FastAPI Modular Monolith
- Service Layer
- Repository Layer
- MongoDB Atlas / AWS S3 / Redis / Vector Store
- Gemini/OpenAI / Firebase / Payment Gateway

5.5 Component Explanation

Flutter Mobile App

The Flutter app is the main user-facing application. It allows users to:

- Sign up
- Log in
- Upload reports
- Ask AI questions
- Track goals
- Set reminders
- View dashboard
- Receive notifications

The mobile app should never directly access MongoDB, Redis, S3, or LLM APIs.

React Admin Dashboard

The admin dashboard is used by internal users to:

- Manage users
- View analytics
- Manage content
- Monitor system health
- Review reports if allowed
- Handle support workflows

Admin actions must be protected with stricter authorization and audit logs.

Cloudflare

Cloudflare can provide:

- DNS
- CDN
- SSL/TLS
- DDoS protection
- Edge security

Decision needed:

Use either Cloudflare as the main DNS layer or AWS Route 53 as the main DNS layer. Avoid using both without a clear boundary.

AWS WAF

AWS WAF protects the backend from:

- Common web attacks
- Bad IPs
- Suspicious traffic
- Basic abuse patterns

Application Load Balancer

The ALB distributes traffic across healthy backend containers.

It should use:

- Health checks
- HTTPS listener
- Multi-AZ routing
- Target groups

FastAPI Backend

The FastAPI backend handles:

- Authentication verification
- Business logic
- AI orchestration
- Report metadata
- Goal tracking
- Reminder creation
- Notification logic
- Admin APIs
- Subscription APIs

MongoDB Atlas

MongoDB Atlas is the primary operational database.

It stores:

- User records
- Profiles
- Conversations
- Messages
- Goals

- Reminders
- Notifications
- Report metadata
- Audit logs
- Subscription records

AWS S3

S3 stores files such as:

- PDFs
- Prescription images
- Food images
- Profile images
- Medical documents

MongoDB should store only the S3 object key and metadata.

Redis

Redis is used for fast temporary data.

It should not be used as the main database.

Vector Store

The vector store stores embeddings and document chunks for RAG.

For MVP, MongoDB Atlas Vector Search can be used to keep the system simpler.

Background Workers

Workers handle:

- OCR
- Embeddings
- Report summaries
- Daily summaries
- Reminder dispatch
- Notification delivery
- Email tasks

5.6 Request Flow — AI Chat

Step 1: User logs in using Firebase Auth.

Step 2: Flutter app receives Firebase ID token.

Step 3: User sends a health question to FastAPI.

Step 4: Request passes through Cloudflare, AWS WAF, and ALB.

Step 5: FastAPI verifies Firebase JWT.

Step 6: Backend checks user role, subscription limits, and rate limits.

Step 7: Backend retrieves user profile summary from Redis or MongoDB.

Step 8: Backend retrieves relevant report chunks from vector search.

Step 9: AI Orchestrator builds a prompt using only necessary context.

Step 10: Gemini/OpenAI generates response.

Step 11: Backend stores the conversation in MongoDB.

Step 12: Response is returned to the mobile app.

5.7 Request Flow — Report Upload

Step 1: User selects a medical report in the mobile app.

Step 2: Backend creates a report metadata record in MongoDB.

Step 3: Backend generates pre-signed S3 upload URL.

Step 4: Mobile app uploads file directly to S3.

Step 5: Backend marks report as uploaded.

Step 6: Worker starts OCR processing.

Step 7: Extracted text is chunked.

Step 8: Embeddings are generated.

Step 9: Chunks and embeddings are stored in vector store.

Step 10: Report summary is generated.

Step 11: User is notified when processing is complete.

5.8 Redis Strategy

Redis should store only short-lived, non-critical data.

Good Redis key examples:

```
rate:user:{userId}:ai_chat:{yyyyMMddHHmm}  
profile:{userId}:summary:v1  
lock:reminder:{reminderId}:{scheduledAt}  
idempotency:upload:{userId}:{requestId}  
admin:metrics:daily:{date}  
provider:llm:{provider}:status
```

5.9 What Must Not Be Cached in Redis

Do not cache:

- Full health reports
- Raw OCR text
- Full AI prompts containing medical data
- Full AI responses containing sensitive medical history
- Access tokens
- Refresh tokens
- Payment card data
- Cross-user RAG results
- Full medical history

Reason:

Redis is a cache. It is not the system of record. Sensitive medical data should stay in MongoDB/S3 with proper access control, encryption, and retention policy.

5.10 High Availability Strategy

Minimum production setup:

- FastAPI containers across at least 2 Availability Zones
- Application Load Balancer health checks
- MongoDB Atlas replica set
- S3 versioning enabled
- Redis ElastiCache with replica in production
- At least 2 background workers
- CloudWatch metrics and logs
- Automated backups

- Restore testing before launch

5.11 Failure Handling

If one FastAPI container fails:

- ALB removes it from rotation.
- Traffic goes to healthy containers.
- New container can be started automatically.

If MongoDB primary fails:

- MongoDB Atlas elects a new primary.
- Backend should use retryable writes and sensible timeouts.

If Redis fails:

- Backend falls back to MongoDB.
- Cache performance reduces.
- Important data is not lost.

If S3 upload fails:

- Upload status remains pending or failed.
- User retries upload.
- No corrupted report record is created.

If LLM provider fails:

- AI chat shows graceful error.
- Normal app features continue working.

If OCR fails:

- Report status becomes `retry_pending` or `processing_failed`.
- Worker retries with backoff.

5.12 Cost Profile

Design 1 has the lowest practical production cost.

Main costs:

- AWS compute
- MongoDB Atlas

- S3 storage
- Redis/ElastiCache
- Firebase usage
- Gemini/OpenAI API usage
- OCR API usage
- Domain and SSL
- Monitoring

5.13 Complexity Profile

Complexity is low to medium.

The team manages:

- One backend application
- One main database
- One object store
- One cache
- Background workers
- External AI services

This is manageable for a small team.

5.14 Strengths

- Fastest to build
- Cheapest to operate
- Easiest to debug
- Best for MVP
- Works well for small team
- Clean internal architecture
- Strong enough for Version 1
- Clear path to future scaling

5.15 Weaknesses

- All modules deploy together
- Scaling is less fine-grained
- AI workload shares backend resources
- Large codebase can become harder to maintain if not structured properly
- Eventually some modules may need extraction

5.16 Best Use Case

Design 1 is best when:

- The product is not yet validated
- The team is small
- Budget matters
- Speed matters
- MVP and Version 1 are the priority
- Stakeholders want a serious but practical launch plan

5.17 Meeting Pitch

“We will launch Blumetara AI with a high-availability modular monolith. This gives us a simple and cost-effective system for MVP and Version 1, while still including serious production concepts such as load balancing, MongoDB replication, S3 file storage, Redis caching, background workers, audit logs, encryption, and disaster recovery.”

6. Design 2 — Hybrid Async Modular Platform

6.1 Purpose

Design 2 is a more advanced architecture that still avoids unnecessary full microservices.

It is designed to support:

- MVP
- Version 1
- Version 2 expansion
- More reliable background processing
- Easier future service extraction
- Higher user growth
- Better failure isolation

This is the recommended growth architecture.

6.2 Core Idea

Design 2 keeps the modular backend, but moves heavy work into durable asynchronous workflows.

Instead of making the user wait for OCR, embeddings, reminders, summaries, and notifications, the backend records the request and publishes events.

6.3 Main Difference from Design 1

Design 1:

FastAPI directly handles most workflows and may trigger workers.

Design 2:

FastAPI handles user requests quickly and pushes heavy work into queues/events.

6.4 High-Level Architecture

Flutter Mobile App / React Admin Dashboard

- Cloudflare
- AWS WAF
- Application Load Balancer
- FastAPI Modular Backend
- Event Layer using SQS/EventBridge
- Worker Fleet
- MongoDB Atlas / S3 / Redis / Vector Store
- Gemini/OpenAI / Firebase / Payment Gateway

6.5 Event Layer

The event layer can include:

- AWS SQS
- AWS EventBridge
- Dead-letter queues
- Retry policies
- Outbox pattern

6.6 Worker Types

Separate workers can handle:

- OCR worker

- Embedding worker
- Report summary worker
- Notification worker
- Reminder worker
- Daily summary worker
- Payment webhook worker
- Analytics worker

6.7 Example Events

ReportUploaded
OCRCompleted
EmbeddingCompleted
ReportSummaryGenerated
ReminderDue
NotificationRequested
PaymentWebhookReceived
DailySummaryRequested
GoalProgressUpdated
SubscriptionUpdated

6.8 Request Flow — Report Upload

Step 1: User uploads report.

Step 2: Backend stores report metadata in MongoDB.

Step 3: Backend stores file in S3 using pre-signed URL.

Step 4: Backend publishes ReportUploaded event.

Step 5: OCR worker consumes ReportUploaded event.

Step 6: OCR worker extracts text.

Step 7: OCR worker publishes OCRCompleted event.

Step 8: Embedding worker consumes OCRCompleted event.

Step 9: Embedding worker chunks text and generates embeddings.

Step 10: Summary worker creates AI summary.

Step 11: Notification worker sends completion notification to user.

6.9 Request Flow — Reminder

Step 1: User creates medicine reminder.

Step 2: Reminder data is stored in MongoDB.

Step 3: Scheduler detects reminder due time.

Step 4: ReminderDue event is published.

Step 5: Notification worker consumes event.

Step 6: Worker sends push notification through FCM.

Step 7: Notification status is stored in MongoDB.

Step 8: Failed delivery is retried.

6.10 Redis Strategy

In Design 2, Redis is still used, but not for durable jobs.

Redis should be used for:

- Rate limiting
- Short-lived cache
- Distributed locks
- Idempotency keys
- Provider health status
- Admin dashboard cache

SQS/EventBridge should be used for:

- OCR jobs
- Embedding jobs
- Notification jobs
- Summary jobs
- Payment webhook jobs
- Reminder jobs

6.11 Why SQS/EventBridge is Better Than Redis for Important Jobs

Redis is fast, but it is not the best durable queue for critical workflows.

SQS/EventBridge provides:

- Better durability
- Retries
- Dead-letter queues
- Message visibility timeout
- Better AWS integration
- Better failure recovery

For a medical tracking app, losing a reminder job or report-processing job is not acceptable. Durable queues are safer.

6.12 High Availability Strategy

Design 2 improves availability because heavy tasks are isolated.

Availability controls:

- Backend across 2+ AZs
- MongoDB Atlas replica set
- S3 versioning
- Redis replication
- SQS durable queues
- Dead-letter queues
- Independent worker scaling
- Worker retries
- Queue depth monitoring
- CloudWatch alarms
- Provider fallback where possible

6.13 Failure Handling

If OCR provider fails:

- Job retries.
- After retry limit, job moves to dead-letter queue.
- User sees report processing delayed, not lost.

If LLM provider fails:

- AI chat degrades.
- Non-AI features continue.
- Provider fallback can be used if configured.

If notification provider fails:

- Notification events remain in queue.
- Worker retries later.
- Failed events go to DLQ.

If worker crashes:

- Message becomes visible again after timeout.
- Another worker can pick it up.

If backend API is under traffic spike:

- Heavy jobs do not block API threads.
- Workers scale separately.

6.14 Cost Profile

Design 2 costs more than Design 1.

Additional costs:

- SQS/EventBridge
- More workers
- More monitoring
- DLQ storage
- Additional logs
- More AWS configuration
- More engineering time

However, the cost is still reasonable compared to full microservices.

6.15 Complexity Profile

Complexity is medium to high.

The team must manage:

- Queues
- Events
- Workers
- Retries
- Idempotency
- Dead-letter queues
- Event schemas

- Async debugging
- Monitoring

6.16 Strengths

- Better reliability
- Better future expansion
- Heavy tasks do not block user requests
- More scalable report processing
- Better notification reliability
- Easier migration to microservices later
- Strong investor story
- Better for Version 2

6.17 Weaknesses

- More complex than Design 1
- More AWS knowledge required
- More monitoring required
- Harder debugging
- More expensive
- Takes longer to build

6.18 Best Use Case

Design 2 is best when:

- The product is expected to grow soon
- Report processing becomes important
- Notifications and reminders must be reliable
- AI workflows become heavier
- Future expansion matters
- Stakeholders want a stronger technical roadmap

6.19 Meeting Pitch

“After the MVP is validated, Blumetara AI can evolve into a hybrid async modular platform. The backend remains clean and modular, but heavy workloads like OCR, embeddings, report summaries, daily summaries, reminders, and notifications move into durable event-driven pipelines. This improves reliability, scalability, and future expansion without forcing us into full microservices too early.”

7. Design 3 — Managed AI Platform with Vertex AI

7.1 Purpose

Design 3 is the premium future architecture.

It is designed for:

- AI-heavy scaling
- Investor roadmap
- Enterprise positioning
- Advanced RAG
- Managed AI infrastructure
- Advanced OCR
- Future AI agents
- Model monitoring and evaluation

This design may be more expensive, but it gives the strongest AI platform story.

7.2 Core Idea

Instead of managing all AI workflows manually through only backend code and basic APIs, Blumetara AI can use a managed AI platform such as Google Vertex AI.

This can support:

- Gemini models
- Embeddings
- Vector Search
- Document AI OCR
- AI model evaluation
- Prompt/version management
- AI monitoring
- Future AI agents

7.3 Two Possible Approaches

There are two ways to use Vertex AI.

Option 3A — AWS Application Layer + Google Vertex AI Layer

In this option:

- Main app stays on AWS.
- FastAPI stays on AWS.
- MongoDB Atlas stays as primary database.
- S3 stores files.
- AI workloads use Google Vertex AI.

Flow:

Flutter/React

- AWS Backend
- MongoDB/S3
- Vertex AI for AI/OCR/Embeddings
- Response to backend
- Response to user

This is a multi-cloud architecture.

Option 3B — Full Google Cloud AI-Native Architecture

In this option:

- Backend runs on Google Cloud Run or GKE.
- Cloud Storage replaces or complements S3.
- Cloud Load Balancer is used.
- Cloud Armor replaces AWS WAF.
- Pub/Sub handles events.
- Vertex AI handles Gemini, embeddings, vector search.
- Document AI handles OCR.
- Firebase fits naturally into the Google Cloud ecosystem.

This is cleaner from an AI platform perspective but requires a bigger cloud shift.

7.4 High-Level Architecture

Flutter Mobile App / React Admin

- Cloudflare or Google Cloud Load Balancer

- Cloud Armor
- FastAPI on Cloud Run or GKE
- AI Orchestrator
- Vertex AI Gemini
- Vertex AI Embeddings
- Vertex AI Vector Search
- Document AI OCR
- MongoDB Atlas / Cloud Storage / S3
- Firebase Auth/FCM

7.5 AI Components

Vertex AI Gemini

Used for:

- AI health chat
- Report explanation
- Daily summaries
- Goal suggestions
- Workout/nutrition personalization
- Health profile-based responses

Vertex AI Embeddings

Used to generate embeddings for:

- Medical report chunks
- Prescription text
- User health profile summaries
- Goal summaries
- Historical health data

Vertex AI Vector Search

Used for:

- Fast retrieval of relevant health report chunks
- RAG
- Similarity search
- Personalized answer context

Document AI

Used for:

- Medical report OCR
- Prescription text extraction
- Structured document parsing
- PDF/image understanding

AI Evaluation

Used for:

- Testing prompt quality
- Monitoring hallucination risk
- Comparing model outputs
- Measuring response quality

7.6 Request Flow — AI Chat

Step 1: User asks a question.

Step 2: Backend verifies authentication.

Step 3: Backend retrieves user profile summary.

Step 4: Backend retrieves relevant embeddings from Vertex AI Vector Search.

Step 5: Prompt builder creates a safe prompt.

Step 6: Vertex AI Gemini generates answer.

Step 7: Backend applies guardrails.

Step 8: Response is stored in MongoDB.

Step 9: Response is returned to user.

7.7 Request Flow — Report Processing

Step 1: User uploads report.

Step 2: File is stored in S3 or Cloud Storage.

Step 3: Report metadata is stored in MongoDB.

Step 4: Document AI extracts text.

Step 5: Text is chunked.

Step 6: Vertex AI Embeddings generates embeddings.

Step 7: Chunks are stored in Vertex AI Vector Search.

Step 8: Gemini generates summary.

Step 9: User is notified when report is ready.

7.8 High Availability Strategy

Design 3 can be highly available, but the exact approach depends on whether AWS or GCP is the primary cloud.

If AWS remains primary:

- AWS backend across multiple AZs
- MongoDB Atlas multi-AZ
- S3 durability
- Google Vertex AI managed availability
- Multi-cloud monitoring
- Strong network/security configuration

If GCP becomes primary:

- Cloud Run multi-zone managed infrastructure
- Cloud Load Balancing
- Cloud Storage durability
- Pub/Sub durability
- Vertex AI managed services
- Firebase native integration
- Google Cloud Monitoring

7.9 Security Considerations

Design 3 requires extra security planning because it may involve multiple cloud providers.

Security requirements:

- Strict IAM permissions
- Service account isolation
- Private networking where possible
- Encrypted data transfer
- No unnecessary PHI sent to AI models

- Vendor agreements
- Data residency review
- Audit logs across cloud providers
- Centralized monitoring

7.10 Cost Profile

Design 3 is the most expensive.

Cost drivers:

- Vertex AI model calls
- Embeddings
- Vector Search
- Document AI OCR
- Cloud Run/GKE
- Pub/Sub
- Storage
- Multi-cloud data transfer
- Monitoring
- Engineering setup time

7.11 Complexity Profile

Complexity is high.

The team must understand:

- AWS
- Google Cloud
- Vertex AI
- Document AI
- IAM
- Multi-cloud networking
- AI monitoring
- Prompt evaluation
- RAG quality
- Cost optimization

7.12 Strengths

- Strongest AI roadmap
- Best managed AI tooling

- Better OCR options
- Better embedding/vector search infrastructure
- Strong investor appeal
- Easier future AI experimentation
- Better for AI agents
- Strong enterprise positioning

7.13 Weaknesses

- Most expensive
- Most complex
- Not ideal for first MVP
- Requires stronger cloud/AI expertise
- Multi-cloud debugging can be difficult
- More vendor dependency
- More compliance review needed

7.14 Best Use Case

Design 3 is best when:

- Product is validated
- Funding is available
- AI quality becomes the main differentiator
- Report processing volume grows
- Advanced RAG is needed
- Investor-facing AI roadmap matters
- Team has enough cloud/AI expertise

7.15 Meeting Pitch

“In the long term, Blumetara AI can evolve into a managed AI platform using services such as Vertex AI, Gemini, Document AI, managed embeddings, and vector search. This gives us a premium AI infrastructure path for advanced report understanding, personalized health intelligence, model evaluation, and future AI agents. Because this is more expensive and complex, it should be treated as a future roadmap after MVP validation.”

8. Side-by-Side Tradeoff Comparison

Criteria	Design 1: Simple HA Modular Monolith	Design 2: Hybrid Async Modular Platform	Design 3: Managed AI Platform
Best For	MVP + Version 1	MVP + Version 1 + Version 2 expansion	Premium AI-heavy future
Build Speed	Fastest	Medium	Slowest
Cost	Lowest	Medium	Highest
Complexity	Low to Medium	Medium to High	High
Team Size	1–3 developers	2–5 developers	4+ developers with AI/cloud experience
MVP Suitability	Excellent	Good	Not ideal
Version 1 Suitability	Excellent	Excellent	Too heavy unless funded
Future Scaling	Good	Very Good	Excellent
High Availability	Good	Strong	Strong but complex
Async Processing	Basic workers	Durable queues/events	Managed cloud-native eventing
AI/RAG Maturity	Good enough	Strong	Very strong
Debugging	Easiest	Moderate	Hardest
Investor Appeal	Good	Very Good	Very High
Risk	Low	Medium	High if started too early
Cloud Lock-in	Medium	Medium	High
Operational Burden	Low	Medium	High

9. Final Recommendation

The best strategy is not to pick only one architecture forever. The best strategy is to use the three designs as a staged roadmap.

Launch Architecture

Use:

Design 1 — Simple High-Availability Modular Monolith

Reason:

It is the fastest, cheapest, and safest way to launch MVP + Version 1 while still looking professional and production-ready.

Growth Architecture

Prepare for:

Design 2 — Hybrid Async Modular Platform

Reason:

It gives a clean path to expand into Version 2 without rewriting the entire product.

Future AI Roadmap

Keep as future option:

Design 3 — Managed AI Platform with Vertex AI

Reason:

It is powerful and investor-friendly, but too expensive and complex for the first launch.

10. Recommended Stakeholder Script

Use this script in the meeting:

“Blumetara AI is a medical tracking and AI health assistant platform, so we are not treating it like a basic app. The system needs secure authentication, encrypted file storage, audit logs, high availability, backup strategy, and careful AI handling.

For launch, we recommend Design 1, which is a high-availability modular monolith. This allows us to build MVP and Version 1 quickly while still using professional architecture principles such as load balancing, MongoDB Atlas replication, S3 file storage, Redis caching, background workers, audit logs, and secure AI orchestration.

As the product grows, we can evolve into Design 2, a hybrid async modular platform. In that model, heavy workloads such as OCR, embeddings, report summaries, reminders, and notifications move into durable event-driven pipelines. This improves reliability and makes future expansion easier.

For the long-term AI roadmap, we can consider Design 3, where managed AI platforms like Vertex AI handle advanced OCR, embeddings, vector search, AI evaluation, and future AI agents. This is more expensive, so we should adopt it only after product validation or funding.

The clear recommendation is to launch with Design 1, architect the code so it can evolve into Design 2, and present Design 3 as the future AI scaling roadmap.”

11. Decisions That Still Need to Be Made

Before final architecture diagrams are created, these decisions should be confirmed.

11.1 Compute Platform

Options:

- AWS App Runner
- ECS Fargate
- EC2

Recommendation:

Use App Runner if simplicity matters. Use ECS Fargate if long-term AWS control matters.

11.2 Redis Deployment

Options:

- Docker Redis for development
- AWS ElastiCache for production
- Self-managed Redis on EC2

Recommendation:

Use Docker Redis locally and AWS ElastiCache in production.

11.3 Async Queue

Options:

- Celery + Redis
- AWS SQS
- EventBridge
- Pub/Sub if using GCP

Recommendation:

Use Celery/Redis only for simple MVP jobs. Use SQS/EventBridge for production-grade background workflows.

11.4 Vector Database

Options:

- MongoDB Atlas Vector Search
- Pinecone
- Weaviate
- Vertex AI Vector Search

Recommendation:

Use MongoDB Atlas Vector Search for MVP to keep architecture simple. Evaluate dedicated vector infrastructure later.

11.5 AI Provider

Options:

- Gemini
- OpenAI
- Both through adapter layer
- Vertex AI later

Recommendation:

Use an AI provider adapter layer so the backend is not locked to one provider.

11.6 OCR Provider

Options:

- Google Vision
- Document AI
- AWS Textract
- LLM-based OCR

Recommendation:

Start with a reliable OCR provider. For future medical-document-heavy scaling, evaluate Document AI or AWS Textract.

11.7 Cloud Strategy

Options:

- AWS-first
- GCP-first
- Multi-cloud

Recommendation:

Start AWS-first because the current architecture already uses AWS. Use Vertex AI only later if AI requirements justify it.

12. Glossary

Modular Monolith

A single backend application that is internally divided into clean modules. It is easier to build than microservices but still maintainable.

Microservices

An architecture where each business domain runs as an independent service. It improves scaling but increases complexity.

High Availability

Designing the system so it continues working even if one server, container, or Availability Zone fails.

Availability Zone

A physically separate data center area inside a cloud region. Running services across multiple AZs improves reliability.

Load Balancer

A service that distributes traffic across multiple backend instances.

AWS WAF

AWS Web Application Firewall. It protects web applications from common attacks.

Redis

An in-memory data store used for caching, rate limiting, locks, and short-lived temporary data.

TTL

Time to Live. It defines how long cached data stays before being automatically deleted.

MongoDB Atlas

A managed MongoDB database service.

AWS S3

Object storage service used for storing files such as PDFs, images, and documents.

RAG

Retrieval-Augmented Generation. It retrieves relevant information from documents before sending context to an LLM.

Embeddings

Numerical representations of text used for similarity search.

Vector Search

A search method that finds semantically similar text chunks using embeddings.

OCR

Optical Character Recognition. It extracts text from images or PDFs.

AI Orchestrator

The backend component that decides how to build prompts, retrieve context, call LLMs, apply guardrails, and return responses.

EventBridge

AWS event bus service used for event-driven architecture.

SQS

AWS Simple Queue Service. It is used for reliable background job queues.

Dead-Letter Queue

A queue where failed messages go after retry attempts are exhausted.

Outbox Pattern

A reliability pattern where events are first stored in the database before being published, preventing lost events.

RBAC

Role-Based Access Control. It controls what users can access based on their role.

Audit Log

A record of important actions such as report access, admin actions, login events, and data deletion.

PHI

Protected Health Information. Sensitive health-related user data that must be handled carefully.

Disaster Recovery

A plan to restore the system after major failure, data loss, or cloud outage.

13. Final One-Line Recommendation

Launch Blumetara AI with **Design 1: Simple High-Availability Modular Monolith**, design the codebase so it can evolve into **Design 2: Hybrid Async Modular Platform**, and present **Design 3: Managed AI Platform with Vertex AI** as the long-term premium AI roadmap.